Computerforensik - Malwareanalyse

Dr. Stefan Hahndel

Technische Hochschule Ingolstadt
Fakultät für Elektrotechnik und Informatik

Teil I

Schadcodeanalyse

Standardproblem: Unbekannte Dateien, Beispiel Linux

Gründe für Untersuchung:

- Keine oder unbekannte Extension
- Datei mit bekannter Extension läßt sich nicht öffnen
- Oft nötig für „reverse engineering“
- Häufig zur Untersuchung von Trojanern / Schadprogrammen

Fragestellungen:

- Was sind Funktion und Fähigkeiten des Programms?
- Gibt es Schadcodefunktionen? (Logische Zeitbombe, Backdoor etc.)
- Sie haben den dazugehörigen Source Code. Stimmt der wirklich mit dem Binary in der Funktion überein?
- Erfahrener Programmierer? Rückschlüsse auf Identität ?

Einführung anhand Hello World

```
#include <stdio.h>
int main(){
    printf("`Hello World!\n");

    return 0;
}
```

```
hahndel@chaosnb:> gcc hw.c -o hw
hahndel@chaosnb:> gcc hw.c -o hw
hahndel@chaosnb:> gcc hw.c -g -o hw_debug
hahndel@chaosnb:> strip hw -o hw_stripped
hahndel@chaosnb:> gcc hw.c -static -o hw_static
```

Vergleich der Dateigrößen

```
hahndel@chaosnb:~/fh/itforensik> ls -alh
insgesamt 657K
...  2 hahndel users  184 28. Mai 14:11 .
... 55 hahndel users 8,8K 28. Mai 14:05 ..
...  1 hahndel users 6,5K 28. Mai 14:10 hw_stripped
...  1 hahndel users  11K 28. Mai 14:10 hw
...  1 hahndel users   74 28. Mai 14:06 hw.c
...  1 hahndel users  12K 28. Mai 14:10 hw_debug
...  1 hahndel users 609K 28. Mai 14:11 hw_static
```

Assemblernotation

mit `gcc hw.c -S -o hw.S` erhalten Sie Assembler

```
.file "hw.c"
...
.globl main
.type main, @function
main:
.LFB2:
pushq %rbp
.LCFI0:
movq %rsp, %rbp
.LCFI1:
movl $.LC0, %edi
call puts
...
```

Analysemethoden

Statisch

- Keine Codeausführung
- Nutzung von Analysetools zum Studium des (Assembler-)Codes

Dynamisch

- Codeausführung
- Dabei Monitoring mit Debugger
- Verschiedene Abläufe direkt beobachtbar
- Problem: Abschottung nötig

1. Schritt: md5-Summen festhalten

```
hahndel@chaosnb:> md5sum hw* > md5sum_hw_files.txt
hahndel@chaosnb:> cat md5sum_hw_files.txt
b92bd28faac194007fe46ebf5567d9ea  hw
979effd43f00bf00755c08a8fc50d0c4  hw.c
26def8877e7467da4a2556fd224e739e  hw_debug
9b2410c77ec819ac4b54162ab2be7e07  hw.s
9b2410c77ec819ac4b54162ab2be7e07  hw.S
6e8aa98adcd31401ce70990f9b724be3  hw_static
be6e4748c37101000a24c561e16d04c2  hw_stripped
```

Das file-Kommando

```
hahndel@chaosnb:> file hw hw_*
hw: ELF 64-bit LSB executable,
    AMD x86-64, version 1 (SYSV), for GNU/Linux 2.6.4,
    dynamically linked (uses shared libs),
    for GNU/Linux 2.6.4, not stripped
hw_debug:    ELF 64-bit LSB executable, AMD x86-64,
       version 1 (SYSV), for GNU/Linux 2.6.4,
     dynamically linked (uses shared libs),
     for GNU/Linux 2.6.4, not stripped
hw_static:   ELF 64-bit LSB executable, AMD x86-64,
     version 1 (SYSV), for GNU/Linux 2.6.4,
     statically linked, for GNU/Linux 2.6.4,
     ...
```

Das strings-Kommando (1)

```
/lib64/ld-linux-x86-64.so.2
SuSESuSE
__gmon_start__
libc.so.6
puts
__libc_start_main
GLIBC_2.2.5
...
‘Hello World!
...
GCC: (GNU) 4.1.2 20061115 (prerelease) (SUSE Linux)
GCC: (GNU) 4.1.2 20061115 (prerelease) (SUSE Linux)
...
```

Das strings-Kommando (2)

- Immer `-a` verwenden
- Per Default gelten ASCII-Zeichen ab 4 Char als String, Änderung der Option mit `-n`
- Auch andere Kodierungen (`-e`) ausprobieren
- Nützlich `-tx` liefert hexadezimalen Offset
- ggf. reguläre Ausdrücke mit anderem Tool verwenden
- Unterschiede zwischen Strings mit `diff` aufspüren

Das strings-Kommando mit (3)

```
hahndel@chaosnb:~/fh/itforensik> xxd hw
...
__do_global_ctors_aux
/usr/src/packages/BUILD/glibc-2.5/cc-nptl/csu/crtn.S
hw.c
_GLOBAL_OFFSET_TABLE_
__init_array_end
__init_array_start
_DYNAMIC
data_start
__libc_csu_fini
_start
__gmon_start__
_Jv_RegisterClasses
puts@@GLIBC_2.2.5
...
```

Nutzung eines hexadezimalen Viewers

```
hahndel@chaosnb:~/fh/itforensik> xxd hw
0000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
0000010: 0200 3e00 0100 0000 5004 4000 0000 0000  ..>.....P.@.....
...
0000350: 005f 5f67 6d6f 6e5f 7374 6172 745f 5f00  .__gmon_start__.
0000360: 6c69 6263 2e73 6f2e 3600 7075 7473 005f  libc.so.6.puts._
0000370: 5f6c 6962 635f 7374 6172 745f 6d61 696e  _libc_start_main
0000380: 0047 4c49 4243 5f32 2e32 2e35 0000 0000  .GLIBC_2.2.5....
...
0001020: 400e 6000 0000 0000 0047 4343 3a20 2847  @.`......GCC: (G
0001030: 4e55 2920 342e 312e 3220 3230 3036 3131  NU) 4.1.2 200611
...
```

Anzeigen der Symbole mit nm

```
hahndel@chaosnb:~/fh/itforensik> nm hw
0000000000600e50 d _DYNAMIC
0000000000600fe8 d _GLOBAL_OFFSET_TABLE_
0000000000400604 R _IO_stdin_used
                 w _Jv_RegisterClasses
0000000000600e30 d __CTOR_END__
0000000000600e28 d __CTOR_LIST__
0000000000600e40 d __DTOR_END__
0000000000600e38 d __DTOR_LIST__
00000000004006d0 r __FRAME_END__
0000000000600e48 d __JCR_END__
0000000000600e48 d __JCR_LIST__
0000000000601028 A __bss_start
0000000000601010 D __data_start
00000000004005c0 t __do_global_ctors_aux
...
```

Häufig genutzte Symboltypen in nm

Typ	Bedeutung
A	absoluter Wert
B	Symbol ist in .bss-Sektion (uninitialisierte statische Daten)
D	Initialisierte Daten (.data)
N	Debug Symbol
R	Read Only Data (.rodata)
T	Text/code Sektion (.text)
U	Undefiniertes Symbol
W	Weak Symbol
?	unbekanntes Symbol

Anzeigen mit Zeilennummern

```
hahndel@chaosnb:> nm --line-numbers hw_debug
..600e50 d _DYNAMIC
..600fe8 d _GLOBAL_OFFSET_TABLE_
..400604 R _IO_stdin_used ...libc-2.5/csu/init.c:25
         w _Jv_RegisterClasses
...
..601028 A __bss_start
..601010 D __data_start
...

..400508 T main /home/hahndel/fh/itforensik/hw.c:3
...
```

Anzeige von Shared Objects mit ldd

```
hahndel@chaosnb:> ldd hw
linux-vdso.so.1 =>  (0x00007fff883fd000)
libc.so.6 => /lib64/libc.so.6 (0x00007f0b7fe61000)
/lib64/ld-linux-x86-64.so.2 (0x00007f0b801a3000)
```

- Die libc-Bibliothek Version 6 wird vom dynamischen Linker ld.so automatisch geladen.
- Der dynamische Linkder lässt sich aber auch manuell aufrufen.

ELF dynamic Linker

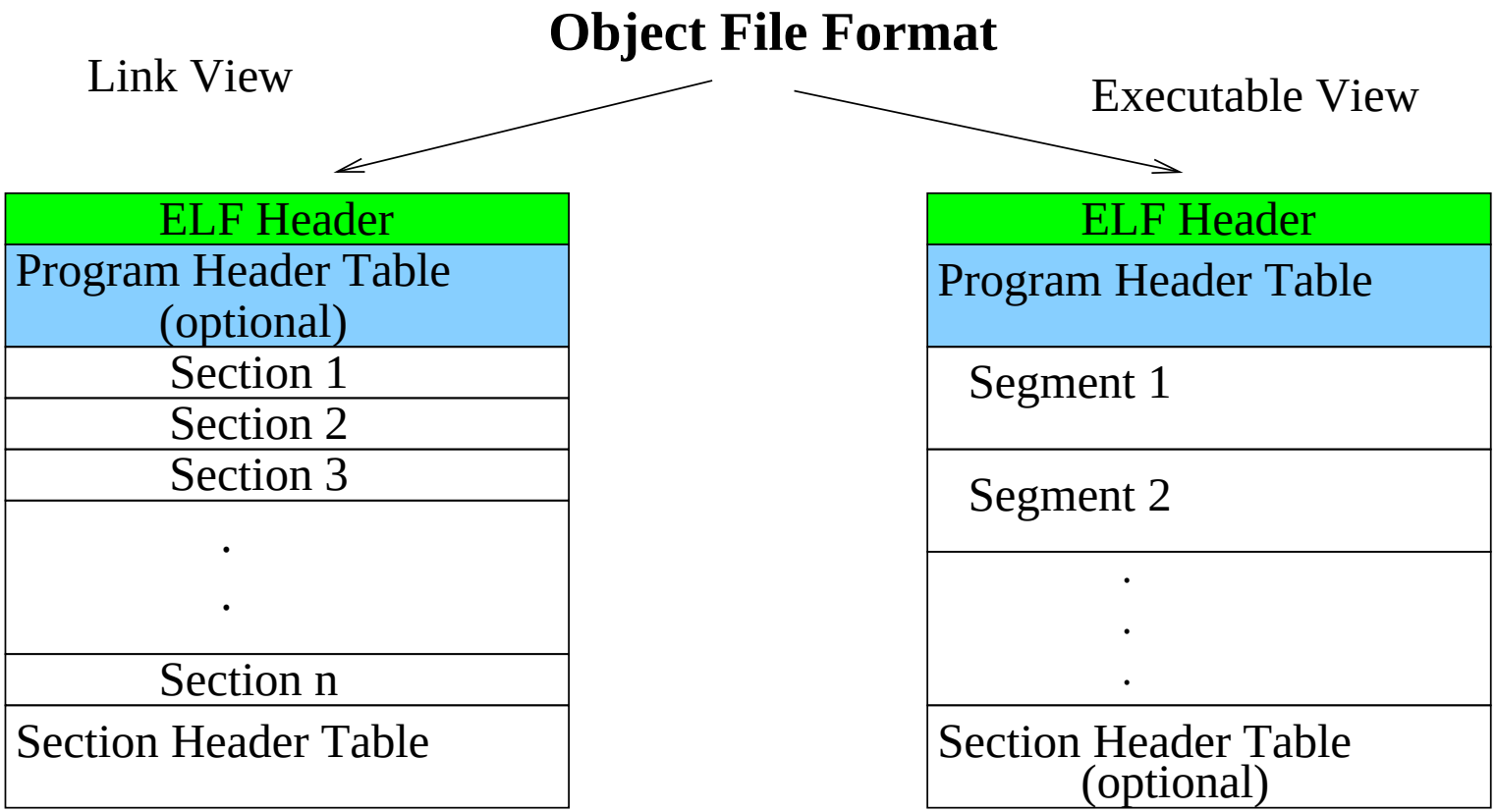
- Der dynamische Linker wird aus Nutzersicht automatisch aufgerufen, sobald nötig
  - Aufgabe: dynamisch Bibliotheken dazubinden
  - d.h. dynamische Bibl. in Speicher laden und Sprungadressen anpassen
- Interessant: Der dynamische Linker kann auch von manuell aufgerufen werden
- Ermöglicht das ausprobieren anderer Versionen
- Information über den dynamischen Linker der aufgerufen werden muss, wird in der `.interp`-Section des Binarys abgelegt.

ELF dynamic Linker - Beschreibung

```
Usage: ld.so [OPTION]... EXECUTABLE-FILE [ARGS-FOR-PROGRAM...]
You have invoked ‘ld.so’, the helper program for shared library executables.
This program usually lives in the file ‘/lib/ld.so’, and special directives
in executable files using ELF shared libraries tell the system’s program
loader to load the helper program from this file.  This helper program loads
the shared libraries needed by the program executable, prepares the program
to run, and runs it.  You may invoke this helper program directly from the
command line to load and run an ELF executable file; this is like executing
that file itself, but always uses this helper program from the file you
specified, instead of the helper program file specified in the executable
file you run.  This is mostly of use for maintainers to test new versions
of this helper program; chances are you did not intend to run this program.

--list                list all dependencies and how they are resolved
--verify              verify that given object really is a dynamically linked
                      object we can handle
--library-path PATH   use given PATH instead of content of the environment
                      variable LD_LIBRARY_PATH
--inhibit-rpath LIST  ignore RUNPATH and RPATH information in object names
                      in LIST
```

ELF Object Datei Format



ELF dynamic Linker - Beschreibung

```
typedef struct
{
  unsigned char e_ident[EI_NIDENT]; /* Magic number and other info */
  Elf64_Half e_type; /* Object file type */
  Elf64_Half e_machine; /* Architecture */
  Elf64_Word e_version; /* Object file version */
  Elf64_Addr e_entry; /* Entry point virtual address */
  Elf64_Off e_phoff; /* Program header table file offset */
  Elf64_Off e_shoff; /* Section header table file offset */
  Elf64_Word e_flags; /* Processor-specific flags */
  Elf64_Half e_ehsize; /* ELF header size in bytes */
  Elf64_Half e_phentsize; /* Program header table entry size */
  Elf64_Half e_phnum; /* Program header table entry count */
  Elf64_Half e_shentsize; /* Section header table entry size */
  Elf64_Half e_shnum; /* Section header table entry count */
  Elf64_Half e_shstrndx; /* Section header string table index */
} Elf64_Ehdr;
```

Identifikation im Hex Editor

```
00: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
10: 0200 3e00 0100 0000 5004 4000 0000 0000  ..>.....P.@.....
20: 4000 0000 0000 0000 3018 0000 0000 0000  @.......0.......
30: 0000 0000 4000 3800 0a00 4000 2600 2300  ....@.8...@.&.#.
40: 0600 0000 0500 0000 4000 0000 0000 0000  ........@.......
...
```

Hier finden wir z.B. Daten über

- Architektur
- Versionsnummer
- Virtuelle Einsprungadresse (Programmstart)
- Start des Section Headers, etc.

Analyse mit readelf

- Wir sehen: Prizipiell Analyse im HexEditor möglich
- Manchmal auch der einzige Weg:
  - z.B. Neue BS-Version ohne entsprechende Tools
  - Verschiedene Unixvarianten mit anderen Formaten
- Exemplarische Vorgehensweise auf andere Systeme übertragbar
- Unter Linux: Einsatz von `readelf` spart uns eine Menge Zeit
- Analyse am Beispiel unserer „HelloWorld“-Anwendung

Elf-Fileheader

readelf –file-header hw liefert:

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x400450
  Start of program headers:          64 (bytes into file)
  Start of section headers:          6192 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  ...
```

ELF-Section-Headers

readelf –section-headers hw liefert:

```
There are 38 section headers, starting at offset 0x1830:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .interp           PROGBITS         0000000000400270  00000270
       000000000000001c  0000000000000000   A       0     0     1
  [ 2] .note.ABI-tag     NOTE             000000000040028c  0000028c
       0000000000000020  0000000000000000   A       0     0     4
  [ 3] .note.SuSE        NOTE             00000000004002ac  000002ac
       0000000000000018  0000000000000000   A       0     0     4
  [ 4] .hash             HASH             00000000004002c8  000002c8
       0000000000000024  0000000000000004   A       5     0     8
  [ 5] .dynsym           DYNSYM           00000000004002f0  000002f0
       0000000000000060  0000000000000018   A       6     1     8
...
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings)
  I (info), L (link order), G (group), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)
  ...
```

ELF Sections (1)

Section	Beschreibung
.bss	Uninitialisierte Daten im Prozessimage
.comment	Info für Versionskontrolle
.data	Initialisierte Daten im Prozessimage
.debug	Debug Info
.dynamic	Info zum dynamischen linken
.dynsym	Symboltabelle fürs dyn. linken
.fini	Code zur Prozessterminierung
.got	Globale Offset Tabelle
.hash	Hashtabelle der Symbole

ELF Sections (1)

Section	Beschreibung
.interp	Name des dynamischen Linkers
.line	Zeilennummern fürs symbolische debuggen
.plt	Prozedur Linkdabelle
.rel<x>	Info für Relocation für Section Nr. x
.rodata	Read-Only Daten
.shstrtab	Section Namen
.strtab	Namen für Symboltabelleneinträge
.symtab	Symboltabelle
.text	ausführbarer Maschinencode

ELF-Programm-Headers (1)

readelf –program-headers hw liefert:

```
Elf file type is EXEC (Executable file)
Entry point 0x400450
There are 10 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000400040 0x0000000000400040
                 0x0000000000000230 0x0000000000000230  R E    8
  INTERP         0x0000000000000270 0x0000000000400270 0x0000000000400270
                 0x000000000000001c 0x000000000000001c  R      1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x0000000000000000 0x0000000000400000 0x0000000000400000
                 0x00000000000006d4 0x00000000000006d4  R E    200000
  LOAD           0x0000000000000e28 0x0000000000600e28 0x0000000000600e28
                 0x0000000000000200 0x0000000000000208  RW     200000
  DYNAMIC        0x0000000000000e50 0x0000000000600e50 0x0000000000600e50
                 0x0000000000000190 0x0000000000000190  RW     8
...
  GNU_EH_FRAME   0x0000000000000618 0x0000000000400618 0x0000000000400618
                 0x0000000000000024 0x0000000000000024  R      4
  GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000  RW     8
  GNU_RELRO      0x0000000000000e28 0x0000000000600e28 0x0000000000600e28
                 0x00000000000001d8 0x00000000000001d8  R      1
...
```

Programm Header und Segmente

- Programm Header
- Gruppierungen von damit zusammenhängenden Segmenten
- „Execution View“ (Sichtweise bei Ausführung)
- Alle Infos, die nötig sind um das Prozessimage zu erzeugen
- Programm Header: Arraymit Einträgen über Typ, Dateistrukturen (fileheader) virtuelle und physikalische Speicheradressen
- Datei und Speicherimagegrösse
- verschiedene Flags
- Infos über alignment

ELF-Programm-Headers (2)

readelf –program-headers hw liefert:

```
Section to Segment mapping:
 Segment Sections...
  00
  01     .interp
  02     .interp .note.ABI-tag .note.SuSE .hash .dynsym .dynstr
  03     .ctors .dtors .jcr .dynamic .got .got.plt .data .bss
  04     .dynamic
  05     .note.ABI-tag
  06     .note.SuSE
  07     .eh_frame_hdr
  08
  09     .ctors .dtors .jcr .dynamic .got
 ...
```

ELF Segment Typen

Typ	Beschreibung
DYNAMIC	Spezifiziert Info fürs dyn. linken (.dynamic)
INTERP	Der dyn. Linker der zu verwenden ist (.interp)
LOAD	Teile der Datei die in den Speicher geladen werden
NOTE	Ort und Grösse von Hilfsinformationen
PHDE	Ort und Grösse der Programm Header Tabelle

ELF-Symboltabellen

readelf –symbols hw liefert:

```
Symbol table '.dynsym' contains 4 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start
     2: 0000000000000000   430 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2
     3: 0000000000000000   421 FUNC    GLOBAL DEFAULT  UND __libc_start

Symbol table '.symtab' contains 80 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
...
    34: 0000000000000000     0 SECTION LOCAL  DEFAULT   34
    35: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS abi-note.S
    36: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS suse-note.S
   ...
```

Debuginformationen

readelf –debug-dump hw liefert:

```
...
The section .debug_info contains:

  Compilation Unit @ offset 0x0:
   Length:        139
   Version:       2
   Abbrev Offset: 0
   Pointer Size:  8
 <0><b>: Abbrev Number: 1 (DW_TAG_compile_unit)
     DW_AT_stmt_list   : 0
     DW_AT_high_pc     : 0x40047c
     DW_AT_low_pc      : 0x40047c
     DW_AT_producer    : (indirect string, offset: 0x0): GNU C 4.1.2 20061115 (prerelease) (SUSE Linux)

     DW_AT_language    : 1 (ANSI C)
     DW_AT_name        : (indirect string, offset: 0x2f): init.c
     DW_AT_comp_dir    : (indirect string, offset: 0x6a): /usr/src/packages/BUILD/glibc-2.5/csu
   ...
```

Hexdump mit readelf

zunächst readelf –section-headers hw liefert:

```
There are 38 section headers, starting at offset 0x1830:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
...
  [ 3] .note.SuSE        NOTE             00000000004002ac  000002ac
       0000000000000018  0000000000000000   A       0     0     4
...
  [13] .text             PROGBITS         0000000000400450  00000450
       00000000000001a4  0000000000000000  AX       0     0     16
  [14] .fini             PROGBITS         00000000004005f4  000005f4
       000000000000000e  0000000000000000  AX       0     0     4
  [15] .rodata           PROGBITS         0000000000400604  00000604
       0000000000000012  0000000000000000   A       0     0     4
  [16] .eh_frame_hdr     PROGBITS         0000000000400618  00000618
       0000000000000024  0000000000000000   A       0     0     4
...
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings)
  I (info), L (link order), G (group), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)
 ...
```

Hexdump mit readelf

readelf –hex-dump=15 hw liefert:

```
Hex dump of section '.rodata':
  0x00400604 646c726f 57206f6c 6c654860 00020001 ....`Hello World
  0x00400614                                 0021 !.
```

readelf –hex-dump=13 hw liefert:

```
Hex dump of section '.text':
  0x00400450 495450f0 e48348e2 89485ed1 8949ed31 1.I..^H..H...PTI
  0x00400460 c7c74800 400530c1 c7480040 0520c0c7 .. .@.H..0.@.H..
  0x00400470 08ec8348 9090f4ff ffffbfe8 00400508 ..@.........H...
  0x00400480 8348d0ff 0274c085 4800200b 59058b48 H..Y. .H..t...H.
  0x00400490 90909090 90909090 90909090 90c308c4 ................
  0x004004a0 9024eb10 74e58948 55000020 0b813d80 .=.. ..UH..t..$.
  0x004004b0 058b48d2 ff00200b 65058948 08c08348 H...H..e. ...H..
  0x004004c0 0b5505c6 e475d285 48108b48 00200b5c \. .H..H..u...U.
  0x004004d0 0000441f 0f660000 441f0fc3 c9010020  ......D..f..D..
  0x004004e0 00b81674 e5894800 0020095f 3d834855 UH.=_. ..H..t...
  0x004004f0 c3894900 600e48bf 0c74c085 48000000 ...H..t..H.`.I..
  0x00400500 400608bf e5894855 9090c3c9 e3ff41c9 .A......UH.....@

...
```

Nutzung von objdump

objdump –section-headers hw liefert:

```
hw:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .interp       0000001c  0000000000400270  0000000000400270  00000270  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.ABI-tag 00000020  000000000040028c  000000000040028c  0000028c  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .note.SuSE    00000018  00000000004002ac  00000000004002ac  000002ac  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .hash         00000024  00000000004002c8  00000000004002c8  000002c8  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .dynsym       00000060  00000000004002f0  00000000004002f0  000002f0  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 .dynstr       0000003d  0000000000400350  0000000000400350  00000350  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .gnu.version  00000008  000000000040038e  000000000040038e  0000038e  2**1
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .gnu.version_r 00000020  0000000000400398  0000000000400398  00000398  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
...
```

Disassemblieren mit objdmp

objdump -d –source ./hw_debug liefert:

```
./hw_debug:     file format elf64-x86-64
...
0000000000400508 <main>:
#include <stdio.h>

int main(){
  400508: 55                    push   %rbp
  400509: 48 89 e5              mov    %rsp,%rbp
printf("‘Hello World!\n");
  40050c: bf 08 06 40 00        mov    $0x400608,%edi
  400511: e8 12 ff ff ff        callq  400428 <puts@plt>

return 0;
  400516: b8 00 00 00 00        mov    $0x0,%eax
}
  40051b: c9                    leaveq
  40051c: c3                    retq
  40051d: 90                    nop
  40051e: 90                    nop
  40051f: 90                    nop
...
```

Dynamische Analyse unseres Programms

- Vor- und Nachteile der statischen Analyse:
  - Anwendung muss nicht gestartet werden: weniger Risiko!
  - Jede Funktionalität kann auf Assemblerebene nachvollzogen werden
  - Grösster Nachteil ist der Zeitaufwand: Die Analyse eines größeren Programms kann sich Wochen oder Monate hinziehen
- Alternative schafft die dynamische Analyse:
  - Vollständige Isolierung von Umgebung nötig!
  - Durchführung in gesicherter Umgebung: Eigene Analyseumgebung (teuer) oder virtuelle Umgebung (vmware, VirtualPC, etc.)
  - Gut vorbereitete Testumgebung mit Debugger und ggf. anderen Systemtools

Systemaufrufe und Signale tracen

Verwendung von strace:

```
execve("./hw", ["./hw"], [/* 96 vars */]) = 0
brk(0)                                  = 0x602000
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fb392666000
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fb392665000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/usr/lib64/tls/x86_64/libc.so.6", O_RDONLY) = -1 ENOENT (No such file or directory)
stat("/usr/lib64/tls/x86_64", 0x7fff9a6644f0) = -1 ENOENT (No such file or directory)
...
...
open("/etc/ld.so.cache", O_RDONLY)      = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=299211, ...}) = 0
mmap(NULL, 299211, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fb39261b000
close(3)                                = 0
open("/lib64/libc.so.6", O_RDONLY)      = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\340\333"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1595305, ...}) = 0
...
...
write(1, "‘Hello World!\n", 14)         = 14
exit_group(0)                           = ?
Process 7076 detached
‘Hello World!
```

Bibliotheksaufrufe tracen

Verwendung von ltrace ./hw:

```
__libc_start_main(0x400508, 1, 0x7fff38e577d8,
            0x400530, 0x400520 <unfinished ...>
puts("\312\215\3458\377\177")           = 14
‘Hello World!
+++ exited (status 0) +++
```

Einige nützliche Optionen:

- Mit `-o` lässt sich die Ausgabe in Datei speichern
- `-x` erlaubt hexadezimale Ausgabe
- `-i` gibt Programmzähler zum Zeitpunkt des Sycalls aus

Verwendung eines Debuggers - Beispiel: gdb

```
(gdb) info functions
All defined functions:

File hw.c:
int main();

Non-debugging symbols:
0x0000000000400400  _init
0x0000000000400428  puts@plt
0x0000000000400438  __libc_start_main@plt
0x0000000000400450  _start
0x000000000040047c  call_gmon_start
0x00000000004004a0  __do_global_dtors_aux
0x00000000004004e0  frame_dummy
0x0000000000400520  __libc_csu_fini
0x0000000000400530  __libc_csu_init
```

Vereinfacht Assemblergrundkenntnisse

- Verwendung einfacher Maschinenbefehle wie z.B. `mov` zum Datentransport
- Die Operanden eines Befehls sind Register, Adressen, Offsets und (direkte) Werte
- Ein Offset ist eine ganze Zahl
- Werte und Adressen werden durch Ausdrücke aus Zahlen und Operatoren dargestellt
- GNU-Assembler `gas` unterstützt die AT&T-Syntax und die Intel-Syntax (Aktivierung Intel-Syntax mit `.intel_syntax`)
- Wir verwenden im folgenden die AT&T-Syntax

Das wichtigste der AT&T-Syntax in Kürze

- Operandenpräfixe: Register werden mit % gekennzeichnet, Direkte Werte mit $, fehlt das Präfix, ist es eine Adresse z.B. `%eax, $42, 0xc00000000000`
- Operandenreihenfolge: Erst kommt der Quell-, dann der Zieloperand (Intel-Syntax ist umgekehrt!), z.B. `mov %eax,30000`
- Befehlssuffixe: Die Operandengrösse von Befehlen wird durch anhängen folgender Suffixe bestimmt: `b` (byte), `w` (16Bit), `l` (32Bit), `q` (64Bit)
- Offsetting: Indirekte Addressierung mit Offset: Speicheradresse steht in Klammern hinter dem Offset

General Purpose Register bei X86_64

64Bit	32Bit	16Bit	obere 8 Bit von 16 Bits	untere 8 Bit
rax	eax	ax	ah	al
rbx	ebx	bx	bh	bl
rcx	ecx	cx	ch	cl
rdx	edx	dx	dh	dl
rsi	esi	si	-	sil
rdi	edi	di	-	dil
rbp	ebp	bp	-	bpl
rsp	esp	sp	-	spl
r8	r8d	r8w	-	r8b
r9	r9d	r9w	-	r9b
...	...	...	...	...
r15	r15d	r15w	-	r15b

Wichtige Spezialregister

- rip ist der Instruktionszeiger (Programm Counter, Instruction Pointer), enthält die Adresse des nächsten auszuführenden Befehls
- rsp ist der Stack Pointer
- rbp wird normalerweise als Frame-Pointer (Zeiger in den Activation Record) benutzt
- rflags Die untersten 16 Bit enthalten die Operation Flags, z.B. carry, zero, sign etc., die nächsten 16 Bit sind aus dem Kernel Mode erreichbar, die oberen 32 Bit (reserviert) liefern immer 0
- Daneben gibt es noch zahlreiche Register für Fliesskommaberechnungen, Media Register für Streamingerweiterungen etc.

Weitere Spezialregister

80Bit (Fließkommaregister)	
	64Bit (MMX overlay)
FPR0	MMX0
FPR1	MMX1
FPR2	MMX2
FPR3	MMX3
FPR4	MMX4
FPR5	MMX5
FPR6	MMX6
FPR7	MMX7

Jeweils 128-Bitregister	
x86	x86_64
XMM0	XMM8
XMM1	XMM9
XMM2	XMM10
XMM3	XMM11
XMM4	XMM12
XMM5	XMM13
XMM6	XMM14
XMM7	XMM15

Speichermodell und Adressierung

- Im 64-Bit-Modus wird ein flaches Adressierungsmodell im Little Endian Format verwendet
- absolute Adressierung: Adressen werden direkt als Werte angegeben
- rip-relative Adressierung: Adressen als Offset zum Instruktionszeiger
- ModR/M-Adressierung: Indirekter Zugriff auf Speicherbereiche, deren Adressen zur Laufzeit berechnet werden
- Daten auf dem Stack werden über den `rsp` adressiert; Veränderung durch Befehle wie `pop`,`push`,`call`,`ret` und `int`

Aufrufkonventionen von Linux bei X86_64

Register	Verwendungszweck	Sicherung
rax	Temporäres Register, erstes Rückgaberegister	Caller
rbx	Callee-gesichertes Register	Callee
rcx	Register für das vierte Argument	Caller
rdx	Register für das dritte Argument, zweites Rückgaberegister	Caller
rsp	Stackpointer	Callee (impl.)
rbp	Callee-gesichertes Register (frame pointer)	Callee
rsi	Register für das zweite Argument	Caller
rdi	Register für das erste Argument	Caller
r8	Register für das fünfte Argument	Caller
r9	Register für das sechste Argument	Caller
r12-15	Callee-gesicherte Register	Callee

Aufrufkonventionen von Windows bei X86_64 (fastcall)

- RCX, RDX, R8, R9: Integer und Zeiger (von links nach rechts)
- XMM0, 1, 2, und 3 das gleiche für Fließkommaargumente
- Zusätzliche Argumente auf dem Stack
- Parameters kürzer als 64 bits long werden nicht mit 0 aufgefüllt (Rest enthält alte Werte)
- Aufrufer muß 32 Byte für RCX,RDX,R8 und R9 allokieren
- Stack wird nach Aufruf durch Aufrufer wiederhergestellt
- Int-Rückgabewert in RAX, Fließkommawert in XMM0, structs auf dem Stack (Zeiger dazu in RCX)
- Stack ist 16-byte aligned.
- Aufrufer legt 8Byte Rücksprungadresse auf Stack
- Beim Allokieren von Speicher auf Stack daher 16n+8 Bytes
- Aufrufer sichert RAX, RCX, RDX, R8, R9, R10, R11
- Funktion für RBX, RBP, RDI, RSI, R12, R13, R14, R15
- Keine Aufrufkonvention für MMX und Fließkommaregister

Überblick wichtiger Assemblerbefehle (1)

Befehl	Erklärung
MOV	Daten zwischen Speicher und Registern transportieren
CMOV*	Verschiedene bedingte Transportbefehle
XCHG	Exchange (Austausch)
BSWAP	Bytes vertauschen
PUSH/POP	Stackbefehle
ADD/ADC	Addition / das gleiche mit Übertrag
SUB/SBC	Subtract/ das gleiche mit Übertrag
MUL/IMUL	Multiplizieren / das gleiche unsigned
DIV/IDIV	Dividieren / das gleiche unsigned
INC/DEC	Increment/Decrement
NEG	Negieren
CMP	Vergleich

Überblick wichtiger Assemblerbefehle (2)

Befehl	Erklärung
AND/OR/XOR/NOT	Bitwise operations
SHR/SAR	Shift right logisch/arithmetisch
SHL/SAL	Shift left logisch/arithmetisch
ROR/ROL	Rotate right/left
RCR/RCL	Rotate right/left inklusive carry bit
BT/BTS/BTR	Bit test/and set/and reset
JMP	Sprung an Adresse
JE/JNE/JC/JNC/J⋆	Sprung falls equal/not equal/carry/not carry/ many others
LOOP/LOOPE/LOOPNE	Loop mit ECX
CALL/RET	Unterprogrammaufruf / return
NOP	No operation
CPUID	CPU information

Einstieg in Assemblercodeanalyse

- Grundidee: MWEs (Minimal Working Examples) in Hochsprache aufbauen
- Dann übersetzen und Ergebnisse disassemblieren
- Sehr schnell werden die Muster in Assembler deutlich
- Auf neuen Architekturen gewinnt man dann auch sehr schnell den Überblick über Übergabemechanismen bei Parametern
- Wir verwenden in den folgenden Beispielen C unter 64bit-Linux
- Führen Sie am Ende ähnliche Experimente auf Ihrer Zielplattform durch!

```c
int main(){
    int i=0;          // <— Assembler ab hier
    if (i==0)
        printf("true\n");
    else
        printf("false\n");
    return 0;
}
```

```
400561: 48 83 ec 10      sub   $0x10,%rsp          ;Speicher f. lokale Var.
400565: c7 45 fc 4*00    movl  $0x0,-0x4(%rbp)     ;int i=0;
40056c: 83 7d fc 00      cmpl  $0x0,-0x4(%rbp)     ;i mit 0 vergleichen
400570: 75 0c            jne   40057e <main+0x21>;ungleich 0: springe zu else
400572: bf 14 06 40 00   mov   $0x400614,%edi      ;    Adr. von true in arg1
400577: e8 c4 fe ff ff   callq 400440 <puts@plt> ;      String ausdrucken
40057c: eb 0a            jmp   400588 <main+0x2b>;else-Zweig überspringen
40057e: bf 19 06 40 00   mov   $0x400619,%edi      ;else: Adr. "false" in arg1
400583: e8 b8 fe ff ff   callq 400440 <puts@plt> ;      String ausdrucken
400588: b8 00 00 00 00   mov   $0x0,%eax           ;return-value 0
40058d: c9               leaveq                    ;
40058e: c3               retq
```

```
int main(){
    int i;                          // <— Assemler ab hier
    for (i=0;i<=255;i++)
        printf("%i ", i);
    return 0;
}
```

```
400561: 48 83 ec 10      sub    $0x10,%rsp
400565: c7 45 fc 4*00    movl   $0x0,-0x4(%rbp)    ; init i=0
40056c: eb 18            jmp    400586 <main+0x29>; goto pruefung
40056e: 8b 45 fc         mov    -0x4(%rbp),%eax
400571: 89 c6            mov    %eax,%esi          ; ab hier:
400573: bf 24 06 40 00   mov    $0x400624,%edi     ; Param f.
400578: b8 00 00 00 00   mov    $0x0,%eax          ; printf
40057d: e8 be fe ff ff   callq  400440 <printf@plt>
400582: 83 45 fc 01      addl   $0x1,-0x4(%rbp)    ; i++
400586: 81 7d fc ff 3*00 cmpl   $0xff,-0x4(%rbp)   ; i<255 ?
40058d: 7e df            jle    40056e <main+0x11>; ja -> Rumpf
40058f: b8 00 00 00 00   mov    $0x0,%eax          ; return 0
400594: c9               leaveq
400595: c3               retq
```

```
int main(){
    int i=0;            // <— Assembler ab hier
    while (i<=255){
        printf("%i ", i);
        i++;
    } return 0;
}
```

```
400561: 48 83 ec 10       sub    $0x10,%rsp
400565: c7 45 fc 4*00     movl   $0x0,-0x4(%rbp)    ; init i=0
40056c: eb 18             jmp    400586 <main+0x29>; goto pruefung
40056e: 8b 45 fc          mov    -0x4(%rbp),%eax
400571: 89 c6             mov    %eax,%esi          ; ab hier:
400573: bf 24 06 40 00    mov    $0x400624,%edi     ; Param f.
400578: b8 00 00 00 00    mov    $0x0,%eax          ; printf
40057d: e8 be fe ff ff    callq  400440 <printf@plt>
400582: 83 45 fc 01       addl   $0x1,-0x4(%rbp)    ; i++
400586: 81 7d fc ff 3*00  cmpl   $0xff,-0x4(%rbp)   ; i<255 ?
40058d: 7e df             jle    40056e <main+0x11>; ja -> Rumpf
40058f: b8 00 00 00 00    mov    $0x0,%eax          ; return 0
400594: c9                leaveq
400595: c3                retq
```

```
int main(){
    int i=0;
    do {
        printf("%i ", i);
        i++;
    } while (i<255);
    return 0;
}
```

```
400561: 48 83 ec 10      sub    $0x10,%rsp
400565: c7 45 fc 4*00    movl   $0x0,-0x4(%rbp)     ;int i=0
40056c: 8b 45 fc         mov    -0x4(%rbp),%eax     ;Schleifenrumpf
40056f: 89 c6            mov    %eax,%esi
400571: bf 24 06 40 00   mov    $0x400624,%edi
400576: b8 4*00          mov    $0x0,%eax
40057b: e8 c0 fe ff ff   callq  400440 <printf@plt>;bis hier
400580: 83 45 fc 01      addl   $0x1,-0x4(%rbp)     ;i++
400584: 81 7d fc fe 3*00 cmpl   $0xfe,-0x4(%rbp)    ;<254 ?
40058b: 7e df            jle    40056c <main+0xf>   ;dann nochmal
40058d: b8 4*00          mov    $0x0,%eax           ;return 0
400592: c9               leaveq
400593: c3               retq
```

```
int i=5,j;
switch (i){    case 1: j=1; break;
               case 2: j=2; break;
               case 3: j=3; break;
               default: j=255;
}
```

```
400501: c7 45 fc 05 3*00 movl   $0x5,-0x4(%rbp)
400508: 8b 45 fc         mov    -0x4(%rbp),%eax
40050b: 83 f8 02         cmp    $0x2,%eax ; case 2:
40050e: 74 13            je     400523 <main+0x26>
400510: 83 f8 03         cmp    $0x3,%eax ; case 3:
400513: 74 17            je     40052c <main+0x2f>
400515: 83 f8 01         cmp    $0x1,%eax ; case 1:
400518: 75 1b            jne    400535 <main+0x38>
40051a: c7 45 f8 01 3*00 movl   $0x1,-0x8(%rbp) ;Aktion 1
400521: eb 19            jmp    40053c <main+0x3f>
400523: c7 45 f8 02 3*00 movl   $0x2,-0x8(%rbp) ;Aktion 2
40052a: eb 10            jmp    40053c <main+0x3f>
40052c: c7 45 f8 03 3*00 movl   $0x3,-0x8(%rbp) ;Aktion 3
400533: eb 07            jmp    40053c <main+0x3f>
400535: c7 45 f8 ff 3*00 movl   $0xff,-0x8(%rbp);default
```

```
int f(int a, int b){ return a+b; }
```

```
4004fd: 55        push %rbp            ;basepointer sichern
4004fe: 48 89 e5  mov  %rsp,%rbp       ;neuer basepointer
400501: 89 7d fc  mov  %edi,-0x4(%rbp);1. Param übernehmen
400504: 89 75 f8  mov  %esi,-0x8(%rbp);2. Param übernehmen
400507: 8b 45 f8  mov  -0x8(%rbp),%eax; Param f. Addition
40050a: 8b 55 fc  mov  -0x4(%rbp),%edx; vorbereiten
40050d: 01 d0     add  %edx,%eax       ; addieren, Res in eax
40050f: 5d        pop  %rbp            ; alter basepointer
400510: c3        retq                 ; zurueck zum Aufrufer
```

```
int main(){
    int i=5, j=10, res;
    res = f(i,j);
    return 0;
}
```

```
400511: 55                  push  %rbp
400512: 48 89 e5            mov   %rsp,%rbp
400515: 48 83 ec 10         sub   $0x10,%rsp
400519: c7 45 fc 05 3*00    movl  $0x5,-0x4(%rbp)  ;init lok.
400520: c7 45 f8 0a 3*00    movl  $0xa,-0x8(%rbp)  ;vars i,j
400527: 8b 55 f8            mov   -0x8(%rbp),%edx
40052a: 8b 45 fc            mov   -0x4(%rbp),%eax
40052d: 89 d6               mov   %edx,%esi        ;Aufruf-
40052f: 89 c7               mov   %eax,%edi        ;param.
400531: e8 c7 ff ff ff      callq 4004fd <f>       ;Aufruf f
400536: 89 45 f4            mov   %eax,-0xc(%rbp)  ;res = ...
400539: b8 00 00 00 00      mov   $0x0,%eax        ;return 0
40053e: c9                  leaveq
40053f: c3                  retq
```

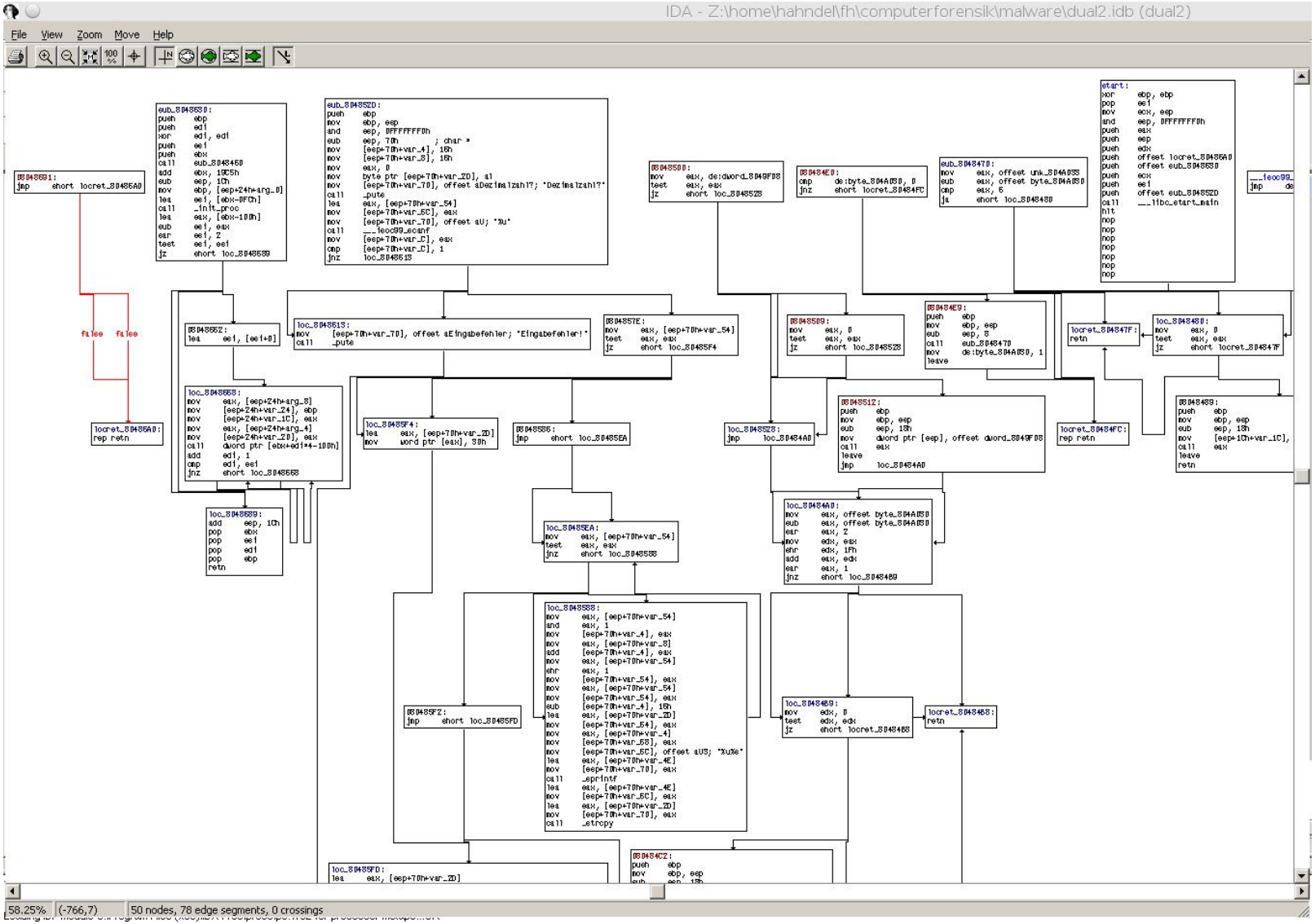
Professionelles Werkzeug: IDA-Pro

```
.bss:0804A030                   assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing
.bss:0804A030 byte_804A030      db ?                    ; DATA XREF: sub_8048470+5↑o
.bss:0804A030                                           ; sub_8048470+1F↑o ...
.bss:0804A031                   db    ? ;
.bss:0804A032                   db    ? ;
.bss:0804A033 unk_804A033       db    ? ;               ; DATA XREF: sub_8048470↑o
.bss:0804A033 _bss              ends
.bss:0804A033
extern:804A034 ; ---------------------------------------------------------------------------
extern:804A034
extern:804A034 ; Segment type: Externs
extern:804A034 ; extern
extern:804A034 ; int printf(const char *,...)
extern:804A034                  extrn printf:near       ; DATA XREF: .got.plt:off_804A00C↑o
extern:804A038 ; char *strcpy(char *,const char *)
extern:804A038                  extrn strcpy:near       ; DATA XREF: .got.plt:off_804A010↑o
extern:804A03C ; int puts(const char *)
extern:804A03C                  extrn puts:near         ; DATA XREF: .got.plt:off_804A014↑o
extern:804A040                  extrn __libc_start_main:near
extern:804A040                                          ; DATA XREF: .got.plt:off_804A01C↑o
extern:804A044 ; int sprintf(char *,const char *,...)
extern:804A044                  extrn sprintf:near      ; DATA XREF: .got.plt:off_804A020↑o
extern:804A048                  extrn __isoc99_scanf:near ; DATA XREF: .got.plt:off_804A024↑o
extern:804A04C                  extrn __gmon_start__:byte ; weak ; DATA XREF: .got:08049FFC↑o
extern:804A04C                                          ; .got.plt:off_804A018↑o
```

```
262144   32 8192 allocating memory for name pointers...
589824          total memory allocated

Loading IDP module C:\Program Files (x86)\IDA Free\procs\pc.w32 for processor metapc...OK
Loading type libraries...
Autoanalysis subsystem has been initialized.
Database for file 'dual2' is loaded.
Compiling file 'C:\Program Files (x86)\IDA Free\idc\ida.idc'...
Executing function 'main'...
```

IDA-Pro - Graphische Darstellung (1)



IDA-Pro - Graphische Darstellung (2)

IDA - Z:\home\hahndel\fh\computerforensik\malware\dual2.idb (dual2)

File View Zoom Move Help

```
sub_8048630:
push    ebp
push    edi
xor     edi, edi
push    esi
push    ebx
call    sub_8048460
add     ebx, 19C5h
sub     esp, 1Ch
mov     ebp, [esp+24h+arg_0]
lea     esi, [ebx-0FCh]
call    _init_proc
lea     eax, [ebx-100h]
sub     esi, eax
sar     esi, 2
test    esi, esi
jz      short loc_8048689
```

```
sub_804852D:
push    ebp
mov     ebp, esp
and     esp, 0FFFFFFF0h
sub     esp, 70h        ; char *
mov     [esp+70h+var_4], 16h
mov     [esp+70h+var_8], 16h
mov     eax, 0
mov     byte ptr [esp+70h+var_2D], al
mov     [esp+70h+var_70], offset aDezimalzahl?; "Dezimalzahl?"
call    _puts
lea     eax, [esp+70h+var_54]
mov     [esp+70h+var_6C], eax
mov     [esp+70h+var_70], offset aU; "%u"
call    ___isoc99_scanf
mov     [esp+70h+var_C], eax
cmp     [esp+70h+var_C], 1
jnz     loc_8048613
```

```
08048500:
mov     eax, ds
test    eax, ea
jz      short lo
```

```
08048662:
lea     esi, [esi+0]
```

```
loc_8048613:
mov     [esp+70h+var_70], offset aEingabefehler; "Eingabefehler!"
call    _puts
```

```
0804857E:
mov     eax, [esp+70h+var_
test    eax, eax
jz      short loc_80485F4
```

```
loc_8048668:
mov     eax, [esp+24h+arg_8]
mov     [esp+24h+var_24], ebp
mov     [esp+24h+var_1C], eax
mov     eax, [esp+24h+arg_4]
mov     [esp+24h+var_20], eax
call    dword ptr [ebx+edi*4-100h]
```

```
0:
```

```
loc_80485F4:
lea     eax, [esp+70h+var_2D]
mov     word ptr [eax], 30h
```

```
08048586:
jmp     short loc_80485EA
```

131.67% (-2071,-38) 50 nodes, 78 edge segments, 0 crossings

IDA-Pro

- Unterstützt zahlreiche Prozessorfamilien
- Unterstützt fast alle wesentlichen Binaryformate
- Professionelle Disassemblierung
  - Assemblerlisting mit Labels
  - Sprunganweisungen und Cross-/Reference graphisch dargestellt
  - Graphische Darstellung des Programmflusses
  - Unterschiedliche Codierungen
  - Umfangreiches Reporting und Exports

Memory Dump Formate bei Nutzung von Volatility

Format	Plugin	Ausgabe
Crash Dump	crashinfo	CPU-Register, kritische Zeiger, Symboltabellen, Speicherauszüge und Code für Fehlerchecks
Hibernation File	hibinfo	CPU register, Zeitstempel, BS Version, Speicherauszüge
HPAK	hpakinfo	Speicherauszüge, Page Filename, Kompression aktiviert/nicht aktiviert
Mach-o	machoinfo	Speicherauszüge
VMware	vmwareinfo	Speicherauszüge, VMX Konfigurationen, CPU-Register, Screenshots (png thumbnails)
Virtual Box	vboxinfo	Speicherauszüge, VirtualBoxversion, Anzahl der CPUs

Allgmeine DLL unter Windows

DLL	Beschreibung
Kernel32.dll	Hauptfunktionalitäten, z.b. Zugriff auf Hardware, Speicher, Files etc. (vergleichbar clib unter Linux
Advapi32.dll	Zugriff zu fortgeschrittenen Windowsfunktionen wie Service Manager und Registry
User32.dll	Userinterfaces wie Buttons, Scrollbars, Reaktion auf Nutzeraktionen
Gdi32.dll	Funktionen zur Manipulation und Darstellung von Graphiken
Ntdll.dll	Interface für Windowskernel. Indirekt durch Kernel32.dll importiert Import durch Nutzerprogramm zeigt, dass Nutzerprogramm Windowsapi umgehen will!
WSock32.dll	Netzwerk DLL
Ws2_32.dll	Netzwerkdienste
Wininet.dll	High Level Netzwerkdienste, z.b. für ftp, http, ntp etc.

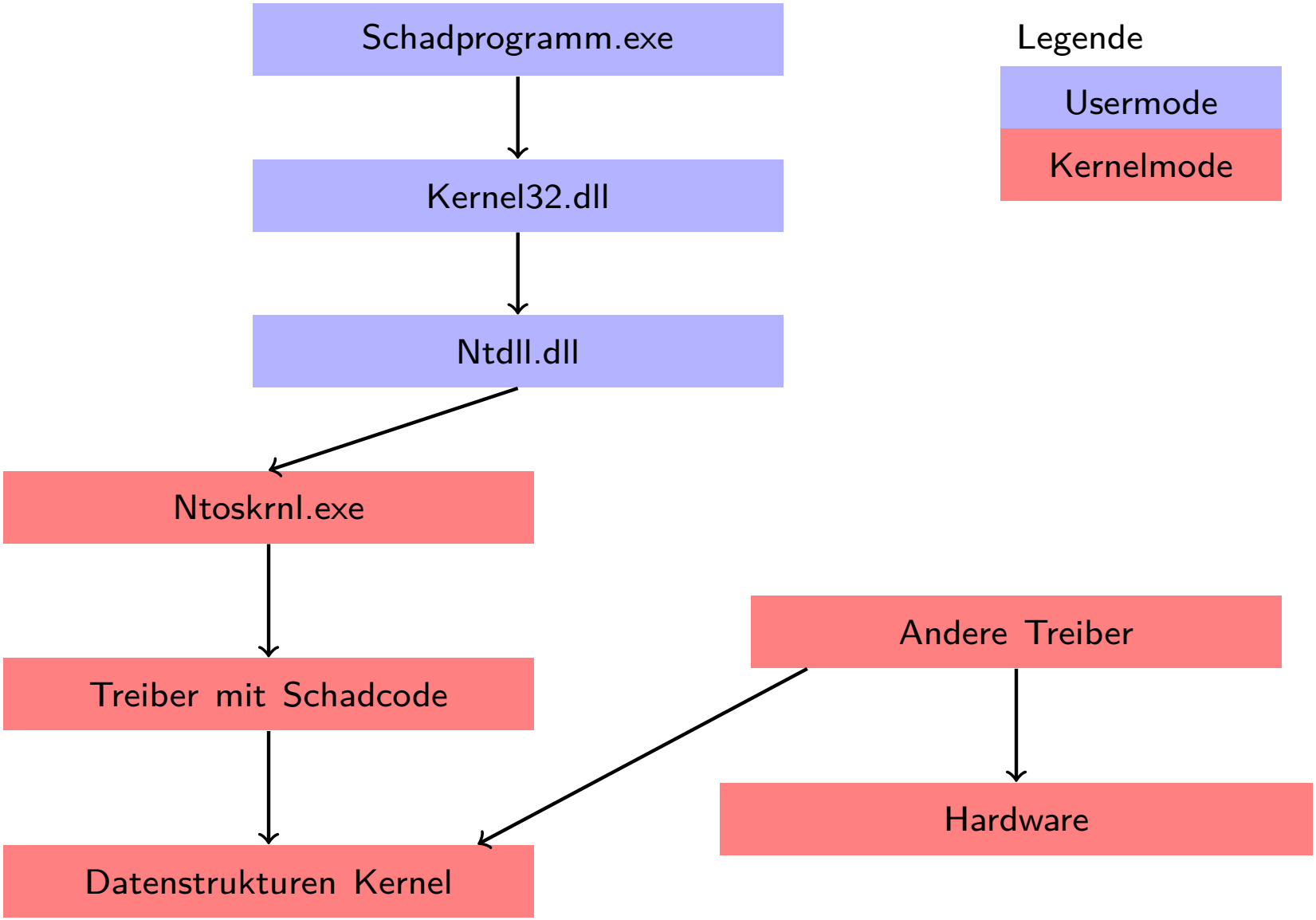
Header und Sections von PE-Files

Section	Beschreibung
.text	Enthält wie bei Linux den Maschinencode, der ausgeführt wird
.rdata	Import und Export Info, erhältlich durch Dependency Walker, PEview, IDA Pro
.data	Globale Daten des Programms
.idata	Infos für Import und Export
.edata	Nicht immer vorhandeni, speichert Exportinfo, falls die Section fehlt, ist die Exportinfo in .idata
.pdata	Nur in 64-Bit-code - Info zum Exception Handling
.rsrc	Ressourcen, die vom Executable benötigt werden
.reloc	Daten zur Relokierung von Bibliotheken

Analyse z.B. mit PEview möglich

Werkzeug Ressource Hacker

- Frei verfügbar
- Zeigt im Binary eingebettete Ressourcen an:
  - Zeichenketten in unterschiedlichsten Kodierungen
  - Icons und Bilder
  - Versionsinformationen etc.
- Für einen ersten Blick recht nützlich!


Schadprogramm.exe
Kernel32.dll
Ntdll.dll
Ntoskrnl.exe
Treiber mit Schadcode
Datenstrukturen Kernel
Andere Treiber
Hardware
Legende
Usermode
Kernelmode

Analysebeispiel - Phishing UPS-Mail

Wichtige Zustellinformationen

Kontrollnummer: 5S64G57508714564

Zustelldatum/-zeit: 09. March 2015 / 12:18:51 vormittags

Sendungsdetails : 5S64G57508714564

Diese E-Mail beinhaltet Informationen, die vertraulich oder nicht für die Öffentlichkeit bestimmt sein könnten. Sollten Sie nicht der designierte Empfänger sein, ist das Lesen, Kopieren, Weiterleiten sowie anderweitiger Gebrauch des Inhalts dieser E-Mail untersagt. Sie werden gebeten, diese E-Mail unverzüglich zu löschen.

Analysebeispiel - Verwendete Tools und Infrastruktur

Physikalisches Betriebssystem

- Grundsystem: Linuxdistribution (hier z.B. : Debian)
- Vorteile der Verwendung von Linux:
  - Geringes Infektionsrisiko durch Malware, da meiste Malware für Windows konzipiert ist
  - Unkompliziertes, wandelbares und hoch modifizierbares System

Verwendete Tools und Infrastruktur

Virtualisierungslösung

- Vorteile der Nutzung einer Virtualisierungslösung:
  - Erstellung von Snapshots zur Wiederherstellung eines bestimmten Systemzustands
  - Schutz des Host-Systems vor Malware
- Nachteil: Moderne Malware erkennt häufig virtuelle Umgebungen
- Virtualisierungslösung: VMWare Workstation
- Zusätzliche Möglichkeiten gegenüber Virtualbox von Oracle

Verwendete Tools und Infrastruktur

Virtuelles Betriebssystem

- Virtuelles Betriebssystem: Windows 7 Pro (32-bit)
- Gründe für die Verwendung von Windows 7:
  - Häufig verwendetes Betriebssystem
  - Meiste Malware ist für Windows konzipiert
- Deaktivierung von UAC, Updates und Firewall für einfachere Umgebung für Malware
- VMWare-Tools wurden nicht installiert, um Erkennung der Umgebung durch Malware zu erschweren
- Diese Konfiguration als Grund-Image verwendet

Tools statische Analyse (1)

- Resource Hacker: Darstellung von Bestandteilen einer ausführbaren Datei unter Windows (z.B. Icons, Menü-Elemente, Strings)
- Dependency Walker: Analyse von Abhängigkeiten in 32-bit und 64-bit Windows-Anwendungen (exe, DLLs) und Darstellung von Import- und Export-Funktionen
- PEView: Anzeige von Informationen über ausführbare Dateien im PE/COFF-Format unter Windows, wie Import- und Exporttabellen, Textsegmente
- RegShot: Sicherung des aktuellen Zustands der Windows-Registry. Später Vergleich mit anderen Zuständen, Identifizierung von veränderten Einträgen durch Malware
- Process Monitor (Aus Sysinternals): Überwachung von Zugriffen aller laufenden Programme auf Festplatte, Netzwerk, Registry; Filterung und Aufzeichnung des Verhaltens von Programmen

Tools statische Analyse (2)

- Process Explorer (Teil der Sysinternal-Suite): Erweiterter Windows Task-Manager; Überprüfung von Signaturen laufender Prozesse, Upload von Prozess-Hashes zu Virustotal, Anzeige der geladenen DLLs eines Prozesses
- Autoruns (Teil der Sysinternal-Suite): Auflistung von Autostart-Einträgen aus verschiedenen Quellen zur Überprüfung unerwünschter Programme beim Systemstart; Deaktivierung von unerwünschten Einträgen möglich
- IDA PRO: Verwendeter Disassembler frei für grundlegende Analysen von X86-32-bit-Anwendungen; Kostenpflichtige Version erforderlich für 64-bit-Malware-Analyse; Ghidra oder Hopper als kostengünstigere Alternative mit ähnlichen Features und OS X/Linux-Unterstützung
- objdump ebenfalls zur Disassemblierung

Hilfsprograme

Für die bessere Auswertung und Bearbeitbarkeit werden ein paar Hilfsprogramme ge- nutzt, die aber mit der eigentlichen Auswertung nichts zu tun haben:

- IZArc (Zip-Programm)
- Notepad++, vim(TextEditor)
- Python (Scriptsprachenumgebung)
- FakeNet (Netzwerkemulator)
- UniversalExtractor (Extrahierer)

Webseiten

- Virustotal:
  - Analyse von Dateien oder URLs, im Original oder als Hash
  - Scannen der Datei/URL durch bekannte Virenscanner
  - Ergebnisse werden zurückgegeben
  - Möglichkeit zum Durchführen breiterer Analysen durch den Kauf weiterer Funktionen
- Malwr:
  - Automatisierte Lösung zur dynamischen Analyse von Malware
  - Basierend auf der Cuckoo-Sandbox
  - Erstellung von Screenshots, Aufzeichnung von Netzwerkverbindungen und Festhalten von Registry-Änderungen
- Immunity Debugger 5:
  - Kostenloser Debugger für Windows
  - Hoher Funktionsumfang und Möglichkeit zur Erweiterung durch Python-Skripte

Infektionsweg

- Analysierte Malware wurde über E-Mail verteilt
- E-Mail-Design orientiert sich am Speditionsunternehmen UPS
- Absender: UPS express [mailto:vorname nachname@smtp.acenet.hu]
- Titel: UPS - Zustellbenachrichtigung, Kontrollnummer 635M6265282635
- Anhang: Zip-Datei mit Namen „up“
- Öffnen des Links in der E-Mail führt zum Download der Zip-Datei

Dynamische Analyse

- Statische Analyse bietet nicht immer ausreichende Informationen über das Verhalten einer Datei.
- Dynamische Analyse ermöglicht die Ausführung eines Programms und die Überprüfung des Verhaltens.
- Analyseumgebung wird verwendet, um die ausführbare Datei auf kompatiblen Systemen auszuführen.
- Durch Ausführung und Überprüfung allein kann das Verhalten nur schwer bestimmt werden.
- Zusätzliche Programme werden während der Ausführung eingesetzt, um Aktionen zu protokollieren.

RegShot: Analyse der Windows Registry

- Windows Registry enthält System- und Programmeinstellungen.
- RegShot ermöglicht das Erstellen von Abbildern der Registry.
- Vergleich der Abbilder zeigt hinzugefügte Einträge.
- Hilfreich, um Veränderungen während der Programmausführung zu erkennen.

Noriben Malware Analysis Sandbox mit SysInternals ProcessMonitor

- Noriben protokolliert geänderte Systemzustände während der Programmausführung.
- Zeigt erstellte/geänderte/gelöschte Dateien und Registry-Einträge.
- In Verbindung mit SysInternals ProcessMonitor für erweiterte Überwachung.

SysInternals: Systemwerkzeuge für Windows

- Sammlung von diversen Programmen für die Analyse von Windows-internen Prozessen.
- Ermöglicht genaue Verfolgung und Überprüfung der Systemaktivitäten.
- Geeignet für die dynamische Analyse von Programmen.

Immunity Debugger: Grafischer Debugger

- Immunity Debugger ermöglicht das Debuggen eines Programms.
- Zeigt Register- und Speicherinhalte während der Ausführung.
- Erlaubt genaue Überprüfung der ausgeführten Schritte.

Wireshark: Analyse des Netzwerkverkehrs

Kennen wir schon aus Netzwerkforensikteil

- Wireshark ermöglicht das Mitschneiden und Analysieren des Netzwerkverkehrs.
- Zeigt Kommunikation mit spezifischen Adressen und übertragene Daten.
- Verschlüsselte Verbindungen erfordern zusätzliche Entschlüsselungsschritte.

Wie schaut die Datei aus

Was der normale Windows-Anwender sieht:

Die vorliegende Datei:

- Dateiname: "VOICE date 08 11 31 03 2015 wav id 039488529348273 lang De".
- Dateiname und Icon lassen auf eine Sounddatei schließen.
- Verwendetes Icon ähnelt einem wav-Icon von Windows.
- Täuscht den Eindruck einer Sounddatei vor.

Personen mit fortgeschrittenen Computerkenntnissen:

- Möglicherweise deaktivieren sie die Option zur Anzeige von Dateiendungen.
- Die Dateiendung ".exe"bleibt verborgen.
- Der Dateityp wird im Explorer als „Anwendung“ angezeigt.

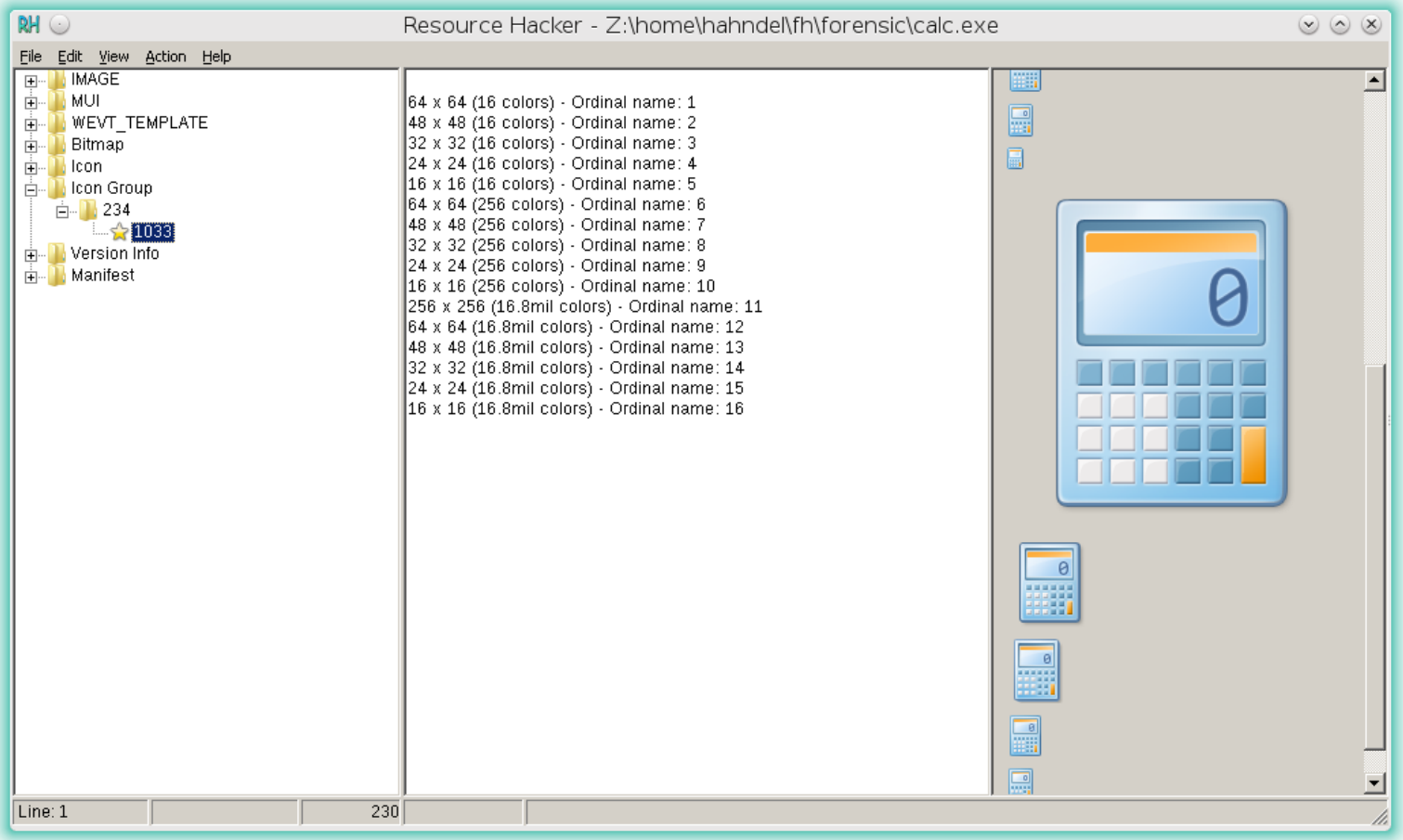
Fazit: Die Datei erscheint auf den ersten Blick nicht als ausführbar. Die Tarnung kann zu einer Opferfalle für Malware werden.

ResourceHacker und PEStudio

- ResourceHacker: Analysiert Ressourcen einer Executable.
  - Ermöglicht die Untersuchung von zusätzlichen Ressourcen in einer ausführbaren Datei.
  - Zum Beispiel Bilder von Icons oder Oberflächenelementen.
- PEStudio: Statische Analyse von ausführbaren Windows Dateien.
  - Zeigt den textuellen Inhalt der Datei an.
  - Bietet eine Bewertungslogik für gefundene Strings in der Datei.
  - Ermöglicht eine relativ einfache Überprüfung der Datei.

Hinweis: Diese Tools bieten keine vollständige Sicherheit, sondern liefern nur Richtwerte und Hinweise.

Werkzeuge: Ressource Hacker



Werkzeuge: PEStudio

pestudio 8.50 - Malware Early Triage - www.winitor.com

File Help

c:\users\cf\malware\analyse\

- Indicators (8/14)
- Virustotal (lookup failed)
- DOS Stub (136 bytes)
- DOS Header (64 bytes)
- File Header (20 bytes)
- Optional Header (224 byte
- Directories (4/15)
- Sections (3)
- Imported libraries (1)
- Imported symbols (2/90)
- Exported symbols (0)
- Exceptions (0)
- Relocations (0)
- Certificates (0)
- Resources (2/6)
- Strings (9/1506)
- Debug (n/a)
- Manifest (n/a)
- Version (1/14)
- Overlay (Unknown)

Type	Size	Blacklisted (9)	Item (1506)
unicode	37	x	\Remounter\Remounter\VeladonACCOJ.vbp
unicode	11	x	Callstb.exe
ascii	5	x	.rsrc
ascii	12	x	MSVBVM60.DLL
ascii	53	x	C:\Program Files\Microsoft Visual Studio\VB98\VB6.OLB
ascii	14	x	CallWindowProc
ascii	8	x	VBA6.DLL
ascii	12	x	MSVBVM60.DLL
ascii	15	x	DllFunctionCall
unicode	5	-	1&'(2
unicode	30	-	@*\AC:\Documents and Settings\
unicode	5	-	Close
unicode	11	-	ScaleHeight
unicode	10	-	ScaleWidth
unicode	4	-	Line
unicode	23	-	gvoYGIdKYUDQqqanOiNmDeD
unicode	12	-	VvCAiQriZmxg
unicode	13	-	goeJEFDInuYzk
unicode	16	-	fsGnErvUGASPmIhn
unicode	10	-	YVbLFMqDCS
unicode	11	-	ZBFTQzMmNVi
unicode	17	-	IhZUxMifCxbdJejrh
unicode	9	-	weSneUscb
unicode	16	-	NfgBCLQuJickaFAB
unicode	8	-	jHfhNTnv
unicode	4	-	&HFF
unicode	7	-	jvuKORI
unicode	5	-	jQFPG
unicode	18	-	cFhRimoaFixthCsixH

Images (1 of 1)

Hilfsprogramme

- IZArc - `http://www.izarc.org/`
- Notepad++ - `https://notepad-plus-plus.org/`
- Python2.7 - `https://www.python.org/download/releases/2.7/`
- FakeNet - `http://practicalmalwareanalysis.com/fakenet/`
- UniversalExtractor - `http://www.legroom.net/software/uniextract`

Statische Analyse

- WinMD5 - `http://winmd5.com/`
- ResourceHacker - `http://www.angusj.com/resourcehacker/`
- PEStudio - `https://winitor.com/`
- IDA - `https://www.hex-rays.com/products/ida/`

Dynamische Analyse

- RegShot - `https://code.google.com/p/regshot/`
- ProcessMonitor - `https://technet.microsoft.com/de-de/sysinternals/bb896645.aspx`
- Noriben Malware Analysis Sandbox - `https://github.com/Rurik/Noriben`
- SysInternals Suite - `https://technet.microsoft.com/de-de/sysinternals/bb545021.aspx`
- Immunity-Dbg - `http://debugger.immunityinc.com/`
- Wireshark - `https://www.wireshark.org/`

Literaturempfehlungen I

Michael Hale Ligh, Andrew Case, Jamie Levy
The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux and Mac Memory - John Wiley & Sons; 1. Auflager, 3. Oktober 2014.

Cameron H. Malin, Eoghan Casey
Linux Malware Incident Response: A Practitioner's Guide to Forensic Collection and Examination of Volatile Data: An Excerpt from Malware Forensic Field Guide for Linux Systems James M. Aquilina, Syngress; Auflage: 1 (12. April 2013)

Michael Sikorski
Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software Andrew Honig, No Starch Press; Auflage: 1 (22. Februar 2012)

Literaturempfehlungen II

- Alexander Geschonneck
  Computer Forensik - dpunkt.verlag, 2. Auflage 2006
- Keith J. Jones, Richard Bejtlich, Curtis W. Rose
  Real Digital Forensics - Addison-Wesley, 2006
- Chad Steel
  Window Forensics - The Field Guide for Conducting Corporate Computer Investigations, Wiley Publishing 2006.
- Dan Farmer, Wietse Venema
  Forensic Discovery. Addison-Wesley, 2005
- Lorenz Kuhlee und Victor Völzow
  Computer Forensik Hacks O´Reilly Hacks Series, 2012.

Literaturempfehlungen III

Harlan Carvey
Perl Scripting for Windows Security: Live Response, Forensic Analysis and Monitoring, Elsevier, 2007.